Міністерство освіти і науки України

Фаховий коледж зварювання та електроніки імені Є.О. Патона

ЗВІТ

З ЛАБОРАТОРНОЇ РОБОТИ №5

навчальної дисципліни

ОПЕРАЦІЙНІ СИСТЕМИ

ФКЗЕ. F2ОС07. 05ЛР

Шифр роботи

Студент Дєтков Н.

Група ПЗ-25-1/9

Перевірив Костенко А.О.

2026

Тема: Написання сценаріїв Windows Script Host.

Мета: ознайомитися з підсистемою Windows Script Host, навчитися писати сценарії мовами VBScript і JScript, працювати з файловою системою через об’єктну модель FileSystemObject, керувати оточенням і ярликами через WScript.Shell та отримувати відомості про систему через WMI.

Обладнання: ПК під керуванням ОС Windows 10 або Windows 11; Windows Script Host; текстовий редактор. Права адміністратора не потрібні.

Варіант: 7.

Хід роботи

1 Отримав у викладача номер індивідуального варіанта 7.

2 Створив на робочому столі каталог LAB5 з підкаталогами src і out командою md LAB5LAB5. Командою copy скопіював у src чотири системні файли з каталогу %SystemRoot%: hosts під назвою hosts.txt, а також networks, protocol і services під назвами networks.txt, protocol.txt і services.txt. Командою echo створив ще чотири текстових файли: file1.txt, file2.txt, file3.txt і readme.txt. Розміри файлів навмисно різні, щоб потім було що відбирати за розміром. Командою dir src перевірив вміст каталогу: у src міститься 8 файлів загальним розміром 20 826 байт, тобто вимогу щонайменше п’яти файлів виконано.

```
C:\...\LAB5>dir src
 Volume in drive C has no label.
 Volume Serial Number is 5A3C-91E2

 Directory of C:\Users\Student\Desktop\LAB5\src

22.10.2026  21:21    <DIR>          .
22.10.2026  21:21    <DIR>          ..
22.10.2026  21:20                14 file1.txt
22.10.2026  21:20                14 file2.txt
22.10.2026  21:20                14 file3.txt
05.10.2026  12:04               824 hosts.txt
07.12.2019  11:12               407 networks.txt
07.12.2019  11:12             1,958 protocol.txt
22.10.2026  21:21               132 readme.txt
07.12.2019  11:12            17,463 services.txt
               8 File(s)         20,826 bytes
               2 Dir(s)  187,342,315,520 bytes free
```

Лістинг 1 - Результат виконання пункту 2

3 Написав сценарій hello.vbs, який приймає аргумент, виводить привітання, ім’я сценарію та повне ім’я хоста. Запустив його через cscript командою cscript

//nologo hello.vbs Студент: вивід з’явився рядками в консолі («Привіт, Студент!», «Сценарій: hello.vbs», «Хост: C:.exe»), ключ //nologo прибрав банер WSH. Потім запустив через wscript командою wscript hello.vbs Студент: кожен виклик WScript.Echo відкрив окреме модальне вікно, яке потрібно закривати клацанням, а в рядку «Хост» вказано wscript.exe. Різниця в тому, що cscript є консольним хостом і виводить результат у вікно cmd, тому підходить для автоматизації та планувальника, а wscript є графічним хостом і показує вивід у вікнах повідомлень, тому підходить для взаємодії з користувачем.

```
C:\...\LAB5>cscript //nologo hello.vbs Студент
Привіт, Студент!
Сценарій: hello.vbs
Хост: C:\WINDOWS\system32\cscript.exe

C:\...\LAB5>wscript hello.vbs Студент

[Вікно «Windows Script Host» № 1]  Привіт, Студент!                          [OK]
[Вікно «Windows Script Host» № 2]  Сценарій: hello.vbs                       [OK]
[Вікно «Windows Script Host» № 3]  Хост: C:\WINDOWS\system32\wscript.exe     [OK]
```

Лістинг 2 - Результат виконання пункту 3

4 Написав сценарій інвентаризації каталогу src inventory.vbs. За допомогою об’єкта Scripting.FileSystemObject сценарій отримує каталог src відносно розташування самого сценарію, обходить колекцію folder.Files і для кожного файлу виводить ім’я, розмір у байтах та дату останньої зміни, а також підсумовує розміри. Результат записується у файл out.txt методом CreateTextFile з третім аргументом True, що вмикає кодування Unicode. Запустив сценарій командою cscript //nologo inventory.vbs і перевірив вміст звіту командою type out.txt: у звіті 8 файлів, підсумковий рядок «Разом: 8 файлів, 20826 байт», що збігається з результатом dir src.

```
C:\...\LAB5>cscript //nologo inventory.vbs
Звіт: out\out.txt

C:\...\LAB5>type out\out.txt
file1.txt       14       22.10.2026 21:20:31
file2.txt       14       22.10.2026 21:20:33
file3.txt       14       22.10.2026 21:20:35
hosts.txt       824      05.10.2026 12:04:19
networks.txt    407      07.12.2019 11:12:44
protocol.txt    1958     07.12.2019 11:12:44
readme.txt      132      22.10.2026 21:21:02
services.txt    17463    07.12.2019 11:12:44
Разом: 8 файлів, 20826 байт
```

Лістинг 3 - Результат виконання пункту 4

5 Реалізував основне завдання варіанта №7: пошук файлів, більших за задану кількість байтів. Написав сценарій findbig.vbs, який за допомогою об’єкта Scripting.FileSystemObject отримує каталог src відносно розташування сценарію, обходить колекцію folder.Files і порівнює властивість file.Size з порогом: якщо розмір строго більший за поріг, ім’я та розмір файлу виводяться, а лічильник знайдених файлів збільшується на одиницю. Поріг у байтах передається першим аргументом (за замовчуванням 1000) і перетворюється на число функцією CLng. Запустив сценарій командою cscript //nologo findbig.vbs 400: знайдено 4 файли (hosts.txt, networks.txt, protocol.txt і services.txt), а readme.txt (132 байти) і три файли по 14 байт не пройшли за порогом. Для перевірки граничного випадку запустив сценарій з порогом, що дорівнює розміру networks.txt (407): файл networks.txt у результат не потрапив, бо порівняння строге («більших за»), тож знайдено 3 файли. Результати збігаються з розмірами, які показала команда dir src.

```
C:\...\LAB5>cscript //nologo findbig.vbs 400
hosts.txt — 824 байт
networks.txt — 407 байт
protocol.txt — 1958 байт
services.txt — 17463 байт
Знайдено файлів: 4

C:\...\LAB5>cscript //nologo findbig.vbs 407
hosts.txt — 824 байт
protocol.txt — 1958 байт
services.txt — 17463 байт
Знайдено файлів: 3
```

Лістинг 4 - Результат виконання пункту 5

6 Додав до сценарію findbig.vbs обробку помилки за схемою з розділу 7 методички. Каталог для пошуку задається другим аргументом (за замовчуванням src). Звернення до каталогу виконується між On Error Resume Next та On Error Goto 0: якщо після виклику fso.GetFolder об’єкт Err містить ненульовий номер помилки, сценарій виводить повідомлення з номером і описом помилки, очищує Err методом Err.Clear і завершується командою WScript.Quit 1. Режим On Error Resume Next увімкнено лише на мінімальній ділянці коду й вимкнено одразу після перевірки. Перевірив код повернення: при запуску cscript //nologo findbig.vbs 400 для існуючого каталогу echo %errorlevel% показала 0, а при зверненні до

неіснуючого каталогу missing сценарій вивів «Помилка 76: Path not found», і echo %errorlevel% показала 1.

```
C:\...\LAB5>cscript //nologo findbig.vbs 400
hosts.txt — 824 байт
networks.txt — 407 байт
protocol.txt — 1958 байт
services.txt — 17463 байт
Знайдено файлів: 4

C:\...\LAB5>echo %errorlevel%
0

C:\...\LAB5>cscript //nologo findbig.vbs 400 missing
Помилка 76: Path not found

C:\...\LAB5>echo %errorlevel%
1
```

Лістинг 5 - Результат виконання пункту 6

7 Переписав мовою JScript сценарій інвентаризації каталогу, створивши inventory.js. Об’єктна модель однакова: використано той самий Scripting.FileSystemObject, але створюється він через new ActiveXObject, а обхід колекції folder.Files виконується за допомогою об’єкта Enumerator (методи atEnd, moveNext, item), тоді як у VBScript достатньо циклу For Each. Запустив сценарій командою cscript //nologo inventory.js: виведено 8 файлів з розмірами та підсумок «Разом: 20826 байт», що збігається з результатом VBScript-версії та з dir src. За обсягом JScript-версія менша (416 байт проти 669 байт у inventory.vbs), але частково через те, що вона не записує звіт у файл і не виводить дати. За читабельністю VBScript простіший завдяки циклу For Each і ключовим словам, схожим на звичайну мову, а JScript потребує явного керування переліком і подвійних зворотних скісних рисок у шляхах.

```
C:\...\LAB5>cscript //nologo inventory.js
file1.txt — 14 байт
file2.txt — 14 байт
file3.txt — 14 байт
hosts.txt — 824 байт
networks.txt — 407 байт
protocol.txt — 1958 байт
readme.txt — 132 байт
services.txt — 17463 байт
Разом: 20826 байт
```

Лістинг 6 - Результат виконання пункту 7

8 Виконав додаткове завдання варіанта №7: вивів п’ять найбільших файлів у порядку спадання. У VBScript немає вбудованого сортування, тому доповнив сценарій findbig.vbs так: знайдені файли тепер не виводяться одразу, а

записуються у два паралельні масиви (імена та розміри), розмір яких задається за кількістю файлів у каталозі через folder.Files.Count. Після обходу масиви впорядковуються за розміром за спаданням методом «бульбашки» із взаємною заміною елементів обох масивів, а виводяться лише перші п’ять елементів (кількість задано константою TOP_COUNT). Якщо знайдено менше п’яти файлів, виводяться всі знайдені, а якщо жодного, сценарій повідомляє про це окремим рядком і завершується з кодом 0. Запустив сценарій командою cscript //nologo findbig.vbs 10: поріг пройшли всі 8 файлів, а з них виведено п’ять найбільших — від services.txt (17463 байти) до readme.txt (132 байти), а три файли по 14 байт у перелік не потрапили. Командою cscript //nologo findbig.vbs 400 перевірив випадок, коли знайдено менше п’яти файлів: виведено всі 4 знайдені. З порогом 20000 жодного файлу не знайдено. Порядок підтвердив командою dir src /O-S, яка сортує каталог за розміром від більшого до меншого: перші п’ять рядків збігаються з виводом сценарію.

```
C:\...\LAB5>cscript //nologo findbig.vbs 10
Пошук файлів, більших за 10 байт, у каталозі src
Знайдено файлів: 8. Найбільші (5):
1. services.txt — 17463 байт
2. protocol.txt — 1958 байт
3. hosts.txt — 824 байт
4. networks.txt — 407 байт
5. readme.txt — 132 байт

C:\...\LAB5>cscript //nologo findbig.vbs 400
Пошук файлів, більших за 400 байт, у каталозі src
Знайдено файлів: 4. Найбільші (4):
1. services.txt — 17463 байт
2. protocol.txt — 1958 байт
3. hosts.txt — 824 байт
4. networks.txt — 407 байт

C:\...\LAB5>cscript //nologo findbig.vbs 20000
Пошук файлів, більших за 20000 байт, у каталозі src
Файлів, більших за 20000 байт, не знайдено.

C:\...\LAB5>dir src /O-S
 Volume in drive C has no label.
 Volume Serial Number is 5A3C-91E2

 Directory of C:\Users\Student\Desktop\LAB5\src

22.10.2026  21:21    <DIR>          .
22.10.2026  21:21    <DIR>          ..
07.12.2019  11:12            17,463 services.txt
07.12.2019  11:12             1,958 protocol.txt
05.10.2026  12:04               824 hosts.txt
07.12.2019  11:12               407 networks.txt
22.10.2026  21:21               132 readme.txt
22.10.2026  21:20                14 file1.txt
22.10.2026  21:20                14 file2.txt
22.10.2026  21:20                14 file3.txt
               8 File(s)         20,826 bytes
               2 Dir(s)  187,342,315,520 bytes free
```

Лістинг 7 - Результат виконання пункту 8

9 Оформив звіт і зробив висновок.

Відповіді на контрольні питання

Високий рівень (творчий)

1. Організація має сотню робочих станцій і потребує щотижневого звіту про вільне місце на дисках. Опишіть, як розв’язати задачу засобами WSH: які об’єкти, як зібрати результати, які обмеження виникнуть.

Для розв’язання задачі засобами WSH основним інструментом буде WMI: сценарій підключається до кожної станції через GetObject(«winmgmts:\» & комп’ютер & «») і виконує запит SELECT DeviceID, Size, FreeSpace FROM Win32_LogicalDisk WHERE DriveType = 3, де умова DriveType = 3 залишає лише локальні жорсткі диски. Перелік комп’ютерів сценарій читає з текстового файлу за допомогою Scripting.FileSystemObject (метод OpenTextFile, рядок за рядком), а результат записує методом CreateTextFile з третім аргументом True, тобто в Unicode, у CSV-файл на мережевій теці. Для відображення імені комп’ютера й користувача додатково можна використати WScript.Network. Зібрати результати можна двома способами. Централізований підхід передбачає один сценарій на сервері, який у циклі обходить усі станції й дописує до спільного файлу рядки «комп’ютер, диск, розмір, вільно, відсоток вільного», а щотижневий запуск забезпечує Планувальник завдань командою cscript //nologo disks.vbs, причому код повернення показує, чи завершився запуск успішно. Розподілений підхід полягає в тому, що той самий сценарій без віддаленого підключення запускається на кожній станції (груповою політикою або сценарієм входу) і записує власний файл з іменем комп’ютера в мережеву теку, що виключає конфлікти одночасного запису. Обмеження при цьому виникають суттєві. Для віддаленого WMI потрібні права адміністратора на станціях, запущена служба WMI та відкриті порти DCOM/RPC у брандмауері. Вимкнені або недоступні комп’ютери спричиняють помилку чи тривалий тайм-аут, тому для кожної станції потрібна окрема обробка помилок (On Error Resume Next на мінімальній ділянці коду та перевірка

Err.Number), а недоступні станції слід фіксувати в звіті окремо. WSH обходить станції послідовно, без паралельності, тож сто станцій потребуватимуть помітного часу. Крім того, розміри сучасних дисків можуть виходити за межі цілого типу VBScript, тому під час обчислень слід використовувати CDbl. Нарешті, оскільки VBScript оголошено застарілим, для нового рішення доцільніше обрати PowerShell з Get-CimInstance Win32_LogicalDisk та Invoke-Command, що підтримує паралельне виконання.

2. Сценарій .vbs — класичний носій шкідливого коду. Поясніть, які саме можливості WSH роблять його небезпечним і якими засобами системи цей ризик обмежують.

Чому небезпечний:

-Сценарій виконується з правами користувача, який його запустив, і не просить підтвердження.

-Файли .vbs і .js асоційовані з WSH, тому запускаються подвійним клацанням, а це легко зробити «випадково» (вкладення в листі, завантажений файл).

-Через CreateObject сценарій має доступ до всієї об’єктної моделі COM: файлової системи (FSO), запуску програм (WScript.Shell.Run/Exec), реєстру, WMI, мережі.

-Це простий текстовий файл без компіляції та встановлення, його легко приховати, змінити й заплутати код, а вбудований у Windows хост є на кожному комп’ютері, тобто додаткових програм зловмиснику не треба.

Чим ризик обмежують:

-Попередження SmartScreen і Mark of the Web для файлів, завантажених із мережі, та блокування таких вкладень поштовими шлюзами.

-Антивірус та інтерфейс AMSI, який дає антивірусу переглянути код сценарію під час виконання.

-Вимкнення WSH політикою або реєстром (HKLMScript Host, параметр Enabled = 0).

-AppLocker, WDAC або політики обмеженого використання програм, що забороняють запуск wscript.exe і cscript.exe або дозволяють лише підписані сценарії.

-Зміна асоціації .vbs на Блокнот, щоб подвійне клацання відкривало файл у редакторі, а не виконувало його (саме цього вимагає застереження на початку лабораторної роботи).

-Принцип найменших привілеїв і UAC.

-Видалення самого VBScript: у Windows 11 24H2 він став компонентом на вимогу, далі його вимкнуть за замовчуванням, а згодом приберуть із системи.

3. Порівняйте WSH і PowerShell за трьома критеріями: доступ до системи, читабельність коду, наявність у сучасних версіях Windows. У яких випадках вибір WSH сьогодні виправданий?

За доступом до системи WSH обмежений COM-об’єктами (FSO, WScript.Shell, WMI): він не має доступу до .NET, готових модулів і вбудованих засобів віддаленого керування. PowerShell працює і з COM, і з повною бібліотекою .NET, має сотні командлетів, постачальників (реєстр, диски як звичайні шляхи) та віддалене виконання через WinRM, тому охоплює значно більше завдань. За читабельністю коду VBScript багатослівний, помилки в ньому обробляються через On Error і Err, а обхід колекцій різниться залежно від мови (For Each у VBScript, Enumerator у JScript), як це було видно при порівнянні inventory.vbs та inventory.js. PowerShell спирається на конвеєр і об’єкти, команди мають передбачувану форму «дієслово-іменник», є повноцінний try/catch, тому запит, який у WSH займає кілька рядків, як у розділі 6 методички, записується одним рядком Get-CimInstance Win32_OperatingSystem. Що стосується наявності у сучасних Windows, WSH присутній у всіх версіях, але VBScript застарілий і поетапно видаляється, тоді як Windows PowerShell 5.1 вбудований у сучасні системи, а PowerShell 7 встановлюється окремо й активно розвивається. Вибір WSH сьогодні виправданий переважно тоді, коли треба підтримувати вже наявні сценарії, які працюють у великій кількості в організації, доки їх не перенесено на PowerShell, коли йдеться про старі чи мінімальні системи, де PowerShell

недоступний або заборонений, або коли робота ведеться зі спадковими COM-компонентами та інструментами, що вимагають саме VBScript. Для нових розробок вибір WSH невиправданий.

4. Той самий алгоритм реалізовано пакетним файлом і сценарієм WSH. Сформулюйте критерії, за якими ви оберете один із варіантів, і проілюструйте кожен критерій прикладом із цієї роботи.

1) Складність логіки та типи даних. Для простих послідовностей команд досить пакетного файлу, наприклад md LAB5LAB5, copy, echo у підготовці каталогів і тестових файлів. Але в findbig.vbs потрібні змінні різних типів, умови, цикл по колекції, порівняння розмірів, лічильник і сортування масивів (If file.Size > limit, бульбашкове сортування), а в пакетному файлі масиви й арифметика незручні.
2) Доступ до об’єктів і властивостей. WSH працює з об’єктами: file.Size, file.DateLastModified, folder.Files.Count в inventory.vbs і findbig.vbs повертають готові значення. У пакетному файлі довелося б розбирати текстовий вивід dir, а порівняння розміру з порогом ускладнювалося б тим, що set /a працює лише із 32-бітними цілими, а dir виводить числа з роздільниками розрядів.
3) Обробка помилок і код повернення. У WSH є Err.Number, Err.Description та WScript.Quit 1. У findbig.vbs звернення до неіснуючого каталогу missing дає повідомлення «Помилка 76: Path not found» і код 1, який перевіряється через echo %errorlevel%. У пакетному файлі доступний лише errorlevel без опису помилки.
4) Кодування та національні символи. Звіт out.txt записано методом CreateTextFile з параметром True у Unicode, і українське «Разом» відображається коректно. Пакетний файл залежить від кодової сторінки консолі, і кирилиця може перетворюватися на нечитабельні символи.
5) Параметризація та режими. findbig.vbs приймає поріг розміру та каталог і розбирає аргументи через WScript.Arguments. У пакетному файлі це громіздко.

6) Простота, швидкість і залежності. Для разової дії або короткої послідовності команд пакетний файл швидший у написанні, не потребує окремого хоста й сумісний з будь-якою версією Windows. Саме так у роботі створено каталоги та тестові файли.
7) Взаємодія між ними. Вибір не завжди або-або: код повернення WSH-сценарію можна перевірити в пакетному файлі або в Планувальнику, тому WSH вбудовують у пакетний файл і навпаки.

Висновок: На цій лабораторній роботі я ознайомився з підсистемою Windows Script Host, навчився писати сценарії мовами VBScript і JScript, працювати з файловою системою через об’єктну модель FileSystemObject (зокрема шукати файли за розміром і виводити п’ять найбільших у порядку спадання), керувати оточенням і ярликами через WScript.Shell та отримувати відомості про систему через WMI.